

OOP and Design Patterns (CSCI 375)
Student Showcase (Final Project) Rubric

1. Project Title: Dungeon-Crawler made using pyGame.

Instructions:

1. There are 10 technical requirements (worth 100 points) to grade the project and the team presentation.
2. For each requirement, use 0 - 10 scale in the Score column (0-6: Fail, 7: C (Average), 8: B (Above Average), 9: A (Good), (10: Excellent)
3. Use the *Notes* section to jot down any observations that may help in grading and justification.

Team and Technical Project Requirement	Score
1. Use 4+1 Views to explain the software design to the audience. <u>Notes:</u> Not the best 4+1 views, logical and process views split between all classes rather than having one to view.	8/10
2. Use of 3 Design Patterns -- presentation clearly stated and briefly explained design patterns use. Common design patterns are Iterator, Decorator, Observer, Strategy, Command, State, Singleton , Adapter, Façade, Flyweight, Abstract Factory, Composite, Template, MVC, etc. <u>Notes:</u> Wrapper class, Handler class, Serialization	10/10
3. Use of Fundamental OOD Concepts- e.g., Inheritance , Abstraction, Getters and Setters , Overloading , Attributes and Methods , etc. <u>Notes:</u> All done well throughout the entire project.	10/10
4. Software management – good usage of management, communication and tracking tools e.g., Gant chart, Kanban board, GitHub Project , Clickup, Discord, Slack, etc. <u>Notes:</u> Utilized to set work for eachother. Communicated weakly through text, calls, and classtime.	9/10
5. Documentation – clear, easy to follow documentation, UML diagrams are complete, and notations are correct; explanation of objects interaction is clear and complete. <u>Notes:</u> Fully documented and accessible with pydoc	10/10

6. Testing and CI-CD pipeline – automatically generates test data using hypothesis, usage of mocking/patching, provides code coverage and Python type check (mypy) reports, CI-CD, Docker, etc. <u>Notes:</u> Full test coverage not achieved (Issues with UI and sound in pygame.)	6/10
7. Clean code and Modularity – project uses appropriate software and system design; the code follows best practices (pep8), is well organized and refactored into packages, modules, classes, etc. <u>Notes:</u> Pep8 guidelines followed throughout the entirety of the project.	9/10
8. Project requirements and execution -- clearly stated functional and technical requirements, project adequately challenging for sophomore-junior students; project demo was clear and concise, etc. <u>Notes:</u> Project demo fully complete, playable with minimal to no bugs.	8/10
9. Team presentation -- all members participated in presentation, used the visual and oral presentation techniques and tools to engage audience, etc. <u>Notes:</u> Little presentation to show, gameplay mostly shown. Oral presentation was the only thing to go off of for grading purposes.	6/10
10. Individual Contributions – briefly explain how and what each member contributed to the successful execution of the project. <u>Notes:</u> Both Betto and Ian contributed greatly to the project. Betto produced visuals and designed entities, items, and UI. Ian designed sound and music, sound managers, and full dungeon generation and asset implementation.	10/10
11. BONUS: Above and beyond – Team went beyond the above list e.g., great User Interface, use of Database, security, real-world application, real-world client delight and interaction, etc. <u>Notes:</u> A fully playable game was developed with a gameplay loop. Not all the goals were reached due to time constraints but the product is complete and doesn't require fixing.	6/10
Total Score <u>Note:</u> Max score can be 110 due to 10 BONUS points.	92/100

